



Las clase NaveEspacial	
NaveEspacial <<Atributos de clase>> maxDeposito=1000 parsec=100 <<Atributos de instancia>> color: char combustible: entero <<Constructores>> NaveEspacial(o: char, c: entero) NaveEspacial(o: char) <<Comandos>> establecerColor(o: char) establecerCombustible(c: entero) llenarDeposito() copy(n:NaveEspacial) <<Consultas>> obtenerColor(): char obtenerCombustible(): entero obtenerAutonomia(): real equals(n:NaveEspacial):boolean clone():NaveEspacial	NaveEspacial(o: char, c: entero) si c > maxDeposito combustible = maxDeposito, en caso contrario asigna c a combustible Requiere el valor de o igual a 'V', 'R' o 'A' NaveEspacial(o: char) combustible = maxDeposito Requiere el valor de o igual a 'V', 'R' o 'A' establecerColor(o: char) Requiere el valor de o igual a 'V', 'R' o 'A' establecerCombustible(c: entero) si c > maxDeposito combustible = maxDeposito. Requiere c > 0. llenarDeposito() combustible = maxDeposito obtenerAutonomia(): real combustible/parsec

El diagrama de la clase NaveEspacial	
NaveEspacial <<Atributos de clase>> maxDeposito=1000 parsec=100 <<Atributos de instancia>> color: char combustible: entero <<Constructores>> NaveEspacial(o: char, c: entero) NaveEspacial(o: char) <<Comandos>> establecerColor(o: char) establecerCombustible(c: entero) llenarDeposito() copy(n:NaveEspacial) <<Consultas>> obtenerColor(): char obtenerCombustible(): entero obtenerAutonomia(): real equals(n:NaveEspacial):boolean clone():NaveEspacial	copy(n:NaveEspacial) Si n está ligado, copia el estado interno del objeto ligado a la variable n, en el objeto que recibe el mensaje, en caso contrario no tiene ningún efecto equals(n:NaveEspacial): boolean Requiere n ligado. Retorna verdadero si el estado interno del objeto que recibe el mensaje es igual al estado interno del objeto ligado a n clone(): NaveEspacial Crea y retorna un nuevo objeto con el mismo estado interno que el objeto que recibe el mensaje

Las clase NaveEspacial	
NaveEspacial <<Atributos de clase>> maxDeposito=1000 parsec=100 <<Atributos de instancia>> color: char combustible: entero <<Constructores>> NaveEspacial(o: char, c: entero) NaveEspacial(o: char) <<Comandos>> establecerColor(o: char) establecerCombustible(c: entero) llenarDeposito() copy(n:NaveEspacial) <<Consultas>> obtenerColor(): char obtenerCombustible(): entero obtenerAutonomia(): real equals(n:NaveEspacial):boolean clone():NaveEspacial	<pre>class NaveEspacial { //Atributos de clase private static final int maxDeposito =1000; private static final int parsec = 100; //Atributos de instancia private char color; private int combustible; }</pre>

Las clase NaveEspacial	
NaveEspacial <<Atributos de clase>> maxDeposito=1000 parsec=100 <<Atributos de instancia>> color: char combustible: entero <<Constructores>> NaveEspacial(o: char, c: entero) NaveEspacial(o: char) <<Comandos>> establecerColor(o: char) establecerCombustible(c: entero) llenarDeposito() copy(n:NaveEspacial) <<Consultas>> obtenerColor(): char obtenerCombustible(): entero obtenerAutonomia(): real equals(n:NaveEspacial):boolean clone():NaveEspacial	<pre>//Constructores public NaveEspacial (char o){ /*combustible = maxDeposito Requiere el valor de o igual a 'V', 'R' o 'A'*/ color = o; combustible = maxDeposito; } Es responsabilidad de la clase cliente garantizar que el primer parámetro tenga valor 'V', 'R' o 'A'</pre>

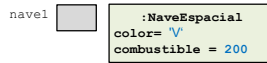
Las clase NaveEspacial	
NaveEspacial <<Atributos de clase>> maxDeposito=1000 parsec=100 <<Atributos de instancia>> color: char combustible: entero <<Constructores>> NaveEspacial(o: char, c: entero) NaveEspacial(o: char) <<Comandos>> establecerColor(o: char) establecerCombustible(c: entero) llenarDeposito() copy(n:NaveEspacial) <<Consultas>> obtenerColor(): char obtenerCombustible(): entero obtenerAutonomia(): real equals(n:NaveEspacial):boolean clone():NaveEspacial	<pre>//Constructores public NaveEspacial (char o){ /*combustible = maxDeposito Requiere el valor de o igual a 'V', 'R' o 'A'*/ color = o; combustible = maxDeposito; } //Constructores public NaveEspacial (char o, int c){ /*si c > maxDeposito combustible = maxDeposito, en caso contrario asigna c a combustible. Requiere el valor de o igual a 'V', 'R' o 'A'*/ color = o; if (c > maxDeposito) combustible = maxDeposito; else combustible = c; }</pre>

La creación de un objeto

```
public NaveEspacial (char o, int c){
    color = o;
    if (c > maxDeposito)
        combustible = maxDeposito;
    else combustible = c;}

```

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
```



La invocación del constructor provoca la creación de un objeto.

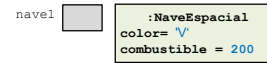
7

El estado interno

```
public NaveEspacial (char o, int c){
    color = o;
    if (c > maxDeposito)
        combustible = maxDeposito;
    else combustible = c;}

```

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
```

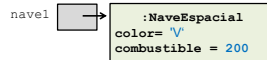


En ejecución, el **estado interno** de un objeto queda definido por los valores de sus atributos de instancia.

8

La referencia

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
```

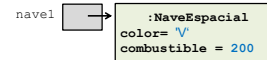


Cuando el constructor termina a la variable `nave1` se le asigna la referencia al objeto.

9

La referencia

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
```

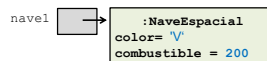


Mientras exista una referencia, el estado interno puede mantenerse en memoria aunque no se esté ejecutando ningún servicio de la clase `NaveEspacial`

10

El valor de una variable de clase

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
```



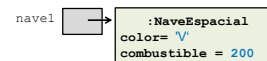
El **valor** de `nave1` es una referencia a un objeto de clase `NaveEspacial`.

11

Los mensajes

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
```

```
float v = nave1.obtenerAutonomia();
```



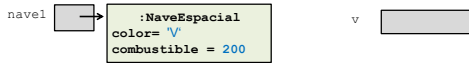
El objeto ligado a `nave1` recibe el mensaje `obtenerAutonomia`.

12

La clase cliente y la clase proveedora

```
public float obtenerAutonomia(){
    //combustible/parsec
    return combustible / parsec;
}
```

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
float v = nave1.obtenerAutonomia();
```



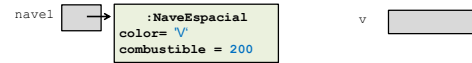
La nave que recibe el mensaje calcula su propia autonomía a partir de su atributo de instancia combustible y el atributo de clase parsec.

13

La clase cliente y la clase proveedora

```
public float obtenerAutonomia(){
    //combustible/parsec
    return combustible / parsec;
}
```

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
float v = nave1.obtenerAutonomia();
```



El método computa y retorna como resultado el cociente.

14

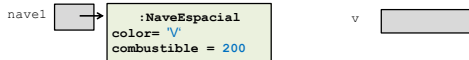
La clase cliente y la clase proveedora

```
public float obtenerAutonomia(){
    //combustible/parsec
    return combustible / parsec;
}
```

Proveedora

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
float v = nave1.obtenerAutonomia();
```

Cliente



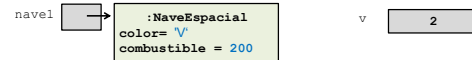
La clase cliente USA los servicios que brinda la clase proveedora.

15

El valor de una variable de tipo elemental

```
public float obtenerAutonomia(){
    //combustible/parsec
    return combustible / parsec;
}
```

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
float v = nave1.obtenerAutonomia();
```



Cuando obtenerAutonomia() termina el valor que retorna se almacena en la variable v.

16

Igualdad y equivalencia

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = new NaveEspacial ('V',200);
```



Las variables nave1 y nave2 mantienen distintas referencias pero están ligados a objetos equivalentes.

17

Igualdad y equivalencia

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = new NaveEspacial ('V',200);
NaveEspacial nave3 = nave2;
```



Los valores de las variables nave2 y nave3 son iguales.

18

Igualdad y equivalencia

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = new NaveEspacial ('V',200);
NaveEspacial nave3 = nave2;
if (nave1 == nave2)
```



La expresión computa false porque nave1 y nave2 mantienen distintas referencias

19

Igualdad y equivalencia

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = new NaveEspacial ('V',200);
NaveEspacial nave3 = nave2;
if (nave3 == nave2)
```



El operador == computa true porque nave3 y nave2 mantienen referencias a una misma nave

20

La identidad

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = new NaveEspacial ('V',200);
NaveEspacial nave3 = nave2;
if (nave3 == nave2)
```



La referencia a un objeto puede utilizarse como su identidad

21

La identidad

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = new NaveEspacial ('V',200);
NaveEspacial nave3 = nave2;
if (nave3 == nave2)
```



nave2 y nave3 tienen la misma identidad.

22

El diagrama de la clase NaveEspacial

NaveEspacial
<<Atributos de clase>> maxDeposito=1000 parsec=100
<<Atributos de instancia>> color: char combustible: entero
<<Constructores>> NaveEspacial(o: char, c: entero) NaveEspacial(o: char)
<<Comandos>> establecerColor(o: char) establecerCombustible(c: entero) llenarDeposito()
copy(n:NaveEspacial)
<<Consultas>> obtenerColor(): char obtenerCombustible(): entero obtenerAutonomia(): real equals(n:NaveEspacial):boolean clone():NaveEspacial

copy(n:NaveEspacial)
Si n está ligado, copia el estado interno del objeto ligado a la variable n, en el objeto que recibe el mensaje, en caso contrario no tiene ningún efecto

El comando copy

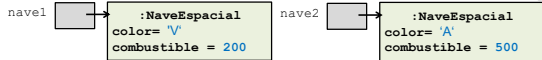
```
NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = new NaveEspacial ('A',500);
```



24

El comando copy

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = new NaveEspacial ('A',500);
nave1.copy(nave2);
```

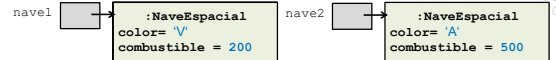


25

El comando copy

```
public void copy(NaveEspacial n){
    /*Si n está ligado, copia el estado interno del objeto ligado a
    la variable n, en el objeto que recibe el mensaje, en caso
    contrario no tiene ningún efecto*/
    //color = n.obtenerColor();
    //combustible = n.obtenerCombustible();
}
```

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = new NaveEspacial ('A',500);
nave1.copy(nave2);
```



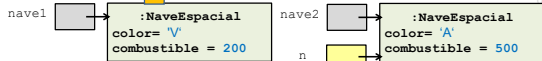
26

El comando copy

```
public void copy(NaveEspacial n){
    /*Si n está ligado, copia el estado interno del objeto ligado a
    la variable n, en el objeto que recibe el mensaje, en caso
    contrario no tiene ningún efecto*/
    //color = n.obtenerColor();
    //combustible = n.obtenerCombustible();
}
```

La variable **n** es un **parámetro formal**, cuando se inicia la ejecución del método se crea una nueva variable y se inicializa con el valor del **parámetro real** nave2.

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = new NaveEspacial ('A',500);
nave1.copy(nave2);
```

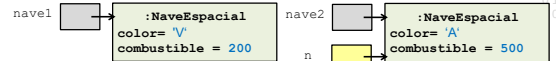


27

El comando copy

```
public void copy(NaveEspacial n){
    /*Si n está ligado, copia el estado interno del objeto ligado a
    la variable n, en el objeto que recibe el mensaje, en caso
    contrario no tiene ningún efecto*/
    if (n != null) {
        //color = n.obtenerColor();
        //combustible = n.obtenerCombustible();
    }
}
```

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = new NaveEspacial ('A',500);
nave1.copy(nave2);
```



28

El parámetro de copy

```
public void copy(NaveEspacial n){
    /*Si n está ligado, copia el estado interno del objeto ligado a
    la variable n, en el objeto que recibe el mensaje, en caso
    contrario no tiene ningún efecto*/
    if (n != null) {
        color = n.obtenerColor();
        combustible = n.obtenerCombustible();
    }
}
```

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = new NaveEspacial ('A',500);
nave1.copy(nave2);
```



29

El parámetro de copy

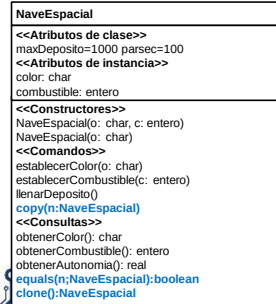
Al terminar la ejecución de **copy** la variable **n** se destruye.

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = new NaveEspacial ('A',500);
nave1.copy(nave2);
```



30

El diagrama de la clase NaveEspacial



equals(n:NaveEspacial): boolean
 Requiere n ligado. Retorna verdadero si el estado interno del objeto que recibe el mensaje es igual al estado interno del objeto ligado a n

La consulta equals

```

boolean b1,b2;
NaveEspacial nave1,nave2;
nave1 = new NaveEspacial ('V',200);
nave2 = new NaveEspacial ('A',500);
    
```

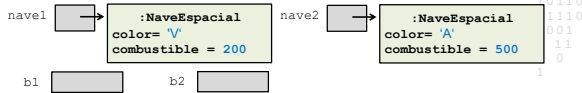


32

La consulta equals

```

boolean b1,b2;
NaveEspacial nave1,nave2;
nave1 = new NaveEspacial ('V',200);
nave2 = new NaveEspacial ('A',500);
b1=nave1.equals(nave2);
    
```

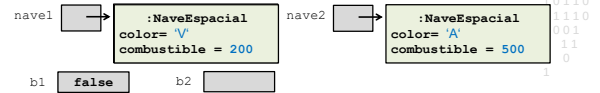


33

La consulta equals

```

boolean b1,b2;
NaveEspacial nave1,nave2;
nave1 = new NaveEspacial ('V',200);
nave2 = new NaveEspacial ('A',500);
b1=nave1.equals(nave2);
    
```



34

La consulta equals

```

boolean b1,b2;
NaveEspacial nave1,nave2;
nave1 = new NaveEspacial ('V',200);
nave2 = new NaveEspacial ('A',500);
b1=nave1.equals(nave2);
nave1.copy(nave2);
    
```



35

La consulta equals

```

boolean b1,b2;
NaveEspacial nave1,nave2;
nave1 = new NaveEspacial ('V',200);
nave2 = new NaveEspacial ('A',500);
b1=nave1.equals(nave2);
nave1.copy(nave2);
b2 = nave1.equals(nave2);
    
```



36

La consulta equals

```

public boolean equals (NaveEspacial n){
    /*Requiere n ligado. Retorna verdadero si el estado interno del
    objeto que recibe el mensaje es igual al estado interno del objeto
    ligado a n*/
    return
}

boolean b1,b2;
NaveEspacial nave1,nave2;
nave1 = new NaveEspacial ('V',200);
nave2 = new NaveEspacial ('A',500);
b1=nave1.equals(nave2);
    
```

Diagram illustrating the state of objects and variables after the first equals call. `nave1` has `color='V'` and `combustible=200`. `nave2` has `color='A'` and `combustible=500`. `b1` is `true` and `b2` is `false`.

La consulta equals

```

public boolean equals (NaveEspacial n){
    /*Requiere n ligado. Retorna verdadero si el estado interno del
    objeto que recibe el mensaje es igual al estado interno del objeto
    ligado a n*/
    return color == n.obtenerColor()
}

boolean b1,b2;
NaveEspacial nave1,nave2;
nave1 = new NaveEspacial ('V',200);
nave2 = new NaveEspacial ('A',500);
b1=nave1.equals(nave2);
    
```

Diagram illustrating the state of objects and variables after the first equals call. `nave1` has `color='V'` and `combustible=200`. `nave2` has `color='A'` and `combustible=500`. `b1` is `true` and `b2` is `false`.

La consulta equals

```

public boolean equals (NaveEspacial n){
    /*Requiere n ligado. Retorna verdadero si el estado interno del
    objeto que recibe el mensaje es igual al estado interno del objeto
    ligado a n*/
    return color == n.obtenerColor() &&
           combustible == n.obteneCombustible();
}

boolean b1,b2;
NaveEspacial nave1,nave2;
nave1 = new NaveEspacial ('V',200);
nave2 = new NaveEspacial ('A',500);
b1=nave1.equals(nave2);
    
```

Diagram illustrating the state of objects and variables after the first equals call. `nave1` has `color='V'` and `combustible=200`. `nave2` has `color='A'` and `combustible=500`. `b1` is `true` and `b2` is `false`.

La consulta equals

```

boolean b1,b2;
NaveEspacial nave1,nave2;
nave1 = new NaveEspacial ('V',200);
nave2 = new NaveEspacial ('V',200);
b1=nave1.equals(nave2);
    
```

Diagram illustrating the state of objects and variables after the second equals call. Both `nave1` and `nave2` have `color='V'` and `combustible=200`. `b1` is `true` and `b2` is `false`.

La consulta equals

```

boolean b1,b2;
NaveEspacial nave1,nave2;
nave1 = new NaveEspacial ('V',200);
nave2 = new NaveEspacial ('V',200);
b1=nave1.equals(nave2);
b2=nave1 == nave2;
    
```

Diagram illustrating the state of objects and variables after the second equals call. Both `nave1` and `nave2` have `color='V'` and `combustible=200`. `b1` is `true` and `b2` is `false`.

El diagrama de la clase NaveEspacial

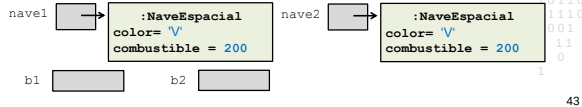
```

classDiagram
    class NaveEspacial {
        <<Atributos de clase>>
        maxDeposito=1000 parsec=100
        <<Atributos de instancia>>
        color: char
        combustible: entero
        <<Constructores>>
        NaveEspacial(o: char, c: entero)
        NaveEspacial(o: char)
        <<Comandos>>
        establecerColor(o: char)
        establecerCombustible(c: entero)
        llenarDeposito()
        copy(n:NaveEspacial)
        <<Consultas>>
        obtenerColor(): char
        obtenerCombustible(): entero
        obtenerAutonomia(): real
        equals(n:NaveEspacial): boolean
        clone(): NaveEspacial
    }
    
```

clone(): NaveEspacial
Crea y retorna un nuevo objeto con el mismo estado interno que el objeto que recibe el mensaje

La consulta clone

```
boolean b1,b2;
NaveEspacial nave1,nave2;
nave1 = new NaveEspacial ('V',200);
nave2 = nave1.clone();
```

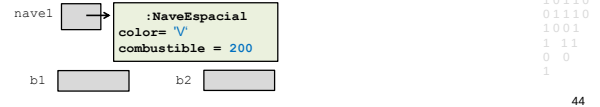


43

La consulta clone

```
public NaveEspacial clone () {
    /*Crea y retorna un nuevo objeto con el mismo estado interno que el
    objeto que recibe el mensaje*/
    return
}

boolean b1,b2;
NaveEspacial nave1,nave2;
nave1 = new NaveEspacial ('V',200);
nave2 = nave1.clone();
```

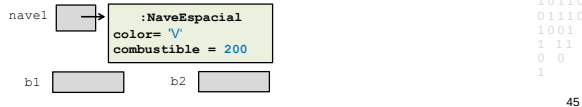


44

La consulta clone

```
public NaveEspacial clone () {
    /*Crea y retorna un nuevo objeto con el mismo estado interno que el
    objeto que recibe el mensaje*/
    return new NaveEspacial
}

boolean b1,b2;
NaveEspacial nave1,nave2;
nave1 = new NaveEspacial ('V',200);
nave2 = nave1.clone();
```

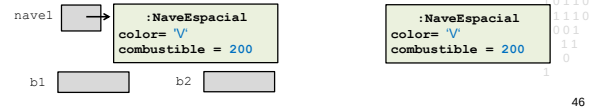


45

La consulta clone

```
public NaveEspacial clone () {
    /*Crea y retorna un nuevo objeto con el mismo estado interno que el
    objeto que recibe el mensaje*/
    return new NaveEspacial(color,combustible);
}

boolean b1,b2;
NaveEspacial nave1,nave2;
nave1 = new NaveEspacial ('V',200);
nave2 = nave1.clone();
```



46

La consulta clone

```
public NaveEspacial clone () {
    /*Crea y retorna un nuevo objeto con el mismo estado interno que el
    objeto que recibe el mensaje*/
    return new NaveEspacial(color,combustible);
}

boolean b1,b2;
NaveEspacial nave1,nave2;
nave1 = new NaveEspacial ('V',200);
nave2 = nave1.clone();
```

El valor que retorna es una referencia a un objeto de clase `NaveEspacial`.



47

La consulta clone

```
public NaveEspacial clone () {
    /*Crea y retorna un nuevo objeto con el mismo estado interno que el
    objeto que recibe el mensaje*/
    return new NaveEspacial(color,combustible);
}

boolean b1,b2;
NaveEspacial nave1,nave2;
nave1 = new NaveEspacial ('V',200);
nave2 = nave1.clone();
```

En cada consulta el tipo que precede al nombre del método es compatible con el tipo del resultado que retorna.



48

El cambio en el diseño

NaveEspacial <<Atributos de clase>> maxDeposito=1000 parsec=100 <<Atributos de instancia>> color: char combustible: entero <<Constructores>> NaveEspacial(color:char,combustible: entero) NaveEspacial(color: char) <<Comandos>> establecerColor(o: char) establecerCombustible(c: entero) llenarDeposito() copy(n:NaveEspacial) <<Consultas>> obtenerColor(): char obtenerCombustible(): entero obtenerAutonomia(): real equals(n:NaveEspacial):boolean clone():NaveEspacial	copy(n:NaveEspacial) Requiere n ligado. Copia el estado interno del objeto ligado a la variable n, en el objeto que recibe el mensaje.
---	--

Es responsabilidad de la clase cliente garantizar que el parámetro está ligado.

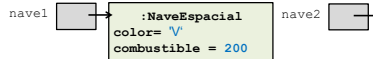
El cambio en la implementación

```
public void copy(NaveEspacial n){
    /*Requiere n ligado. Copia el estado interno del objeto
    ligado a la variable n, en el objeto que recibe el
    mensaje.*/
    color = n.obtenerColor();
    combustible = n.obtenerCombustible();
}
```

El cambio en la implementación

```
public void copy(NaveEspacial n){
    /*Requiere n ligado. Copia el estado interno del objeto
    ligado a la variable n, en el objeto que recibe el
    mensaje.*/
    color = n.obtenerColor();
    combustible = n.obtenerCombustible();
}
```

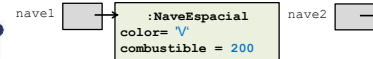
```
NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = null;
```



El cambio en la implementación

```
public void copy(NaveEspacial n){
    /*Requiere n ligado. Copia el estado interno del objeto
    ligado a la variable n, en el objeto que recibe el
    mensaje.*/
    color = n.obtenerColor();
    combustible = n.obtenerCombustible();
}
```

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = null;
if (nave2 != null) nave1.copy(nave2);
```



Los errores de compilación y de ejecución

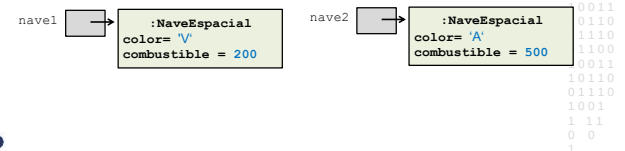
```
public void copy(NaveEspacial n){
    /*Requiere n ligado. Copia el estado interno del objeto
    ligado a la variable n, en el objeto que recibe el
    mensaje.*/
    color = n.obtenerColor();
    combustible = n.obtenerCombustible();
}
```

```
nave1.copy(nave2);
```

¿Qué ocurre si nave1 es una referencia nula?

¿Qué ocurre si nave2 es una referencia nula?

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = new NaveEspacial ('A',500);
```



```

    class NaveEspacial {
    public:
        color = 'A'
        combustible = 500
    }

    navel1 = NaveEspacial()
    nave2 = NaveEspacial()
    nave3 = NaveEspacial()
  
```

The diagram illustrates the memory layout of four objects in memory. The objects are represented as boxes with their attributes and values. The memory addresses are shown on the right side of the diagram.

- nave1** (NaveEspecial): color = 'V', combustible = 200. Address: 0x1010.
- nave2** (NaveEspacial): color = 'A', combustible = 500. Address: 0x1014.
- nave3** (NaveEspecial): color = 'V', combustible = 200. Address: 0x1018.
- nave4** (NaveEspacial): color = 'A', combustible = 500. Address: 0x101C.

The memory layout shows the objects are stored in a contiguous block, with nave1 and nave3 at the bottom and nave2 and nave4 at the top. The memory addresses are 0x1010, 0x1014, 0x1018, and 0x101C.

Diagram illustrating the state of a linked list after inserting a new node. The list contains three nodes: 'navel' (blue), 'nave2' (green), and 'nave3' (blue). Each node is represented by a small colored box with an arrow pointing to a larger box containing its attributes. 'navel' has color 'V' and combustibile 200. 'nave2' has color 'A' and combustibile 250. 'nave3' has color 'V' and combustibile 200. To the right, a vertical list of binary strings shows the state of the list: 0110, 1110, 1100, 0011, 1010, 0111, 1001, 1111, 0100, 0000, 0001.

Recomendaciones

Comiencen YA a leer el material publicado.

En IPOO la "teoría" está integrada a la "práctica", la teoría es necesaria para resolver problemas aplicando los conceptos de la POO y la práctica es necesaria para comprender la utilidad de los conceptos de la POO en términos de calidad y productividad de software.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Recomendaciones

Practiquemos "leer" instrucciones, la comunicación es una capacidad esencial en nuestra disciplina.

```
float v = nave1.obtenerAutonomia();
```

- Declara la variable v de tipo elemental float.
- El objeto ligado a la variable nave1 recibe el mensaje obtenerAutonomía.
- El resultado que retorna en respuesta al mensaje se asigna a la variable v.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Recomendaciones

Practiquemos analizar por qué podría dar error esta instrucción.

```
float v = nave1.obtenerAutonomia();
```

- La variable v ya está declarada dentro del mismo ambiente de referenciamiento.
- La variable nave1 no está ligada a un objeto.
- No existe un método que pueda ligarse al mensaje obtenerAutonomia para el objeto ligado a nave1 o no coincide la signature.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Recomendaciones

Comencemos a pensar qué controles son estáticos y cuáles son dinámicos.

```
float v = nave1.obtenerAutonomia();
```

De los errores mencionados, ¿cuáles puede detectar el compilador?

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```